Implementação e Validação em Hardware do Processador MIPS Monociclo com Instruções de Desvio

Segunda Entrega: Síntese em FPGA Intel MAX 10, Simulação RTL e Verificação Estrutural

José Dercy Vieira da Silva Filho
Vinicius Menezes
Isabela Furtado

Faculdade de Engenharia Elétrica e da Computação – FEEC
Universidade Federal do Amazonas – UFAM
Manaus, Brasil

Arquitetura de Sistemas Digitais e Laboratório – 2026/2

Agenda e Estrutura da Apresentação (Duração: $\approx$ 8 minutos)

Divisão de Tempos e Responsabilidades

- **Bloco 1 (0:00 – 2:30): José Dercy**
  ULA 32 bits, GTKWave (`make waves-alu`) e Quartus RTL
- **Bloco 2 (2:30 – 5:00): Vinicius Menezes**
  Banco de Registradores (32 × 32), Controle e Saltos
- **Bloco 3 (5:00 – 7:30): Isabela Furtado**
  Memórias ROM/RAM, Top-Level e GTKWave (`make waves-top`)
- **Encerramento (7:30 – 8:00): Equipe**
  Síntese FPGA MAX 10, Rubricas e Próximos Passos

AÇÃO DE VÍDEO / GRAVAÇÃO DE TELA

- **0:00**: Câmera com os 3 alunos reunidos (Meet / OBS);
- **0:20**: Dercy compartilha os slides na tela;
- **1:20**: Dercy corta para GTKWave da ULA;
- **1:55**: Dercy corta para Quartus Prime (RTL Viewer);
- **2:30**: Passagem da palavra para Vinicius;
- **5:00**: Passagem da palavra para Isabela;
- **6:40**: Isabela corta para GTKWave do Top-Level;
- **7:30**: Retorno aos slides finais com os 3 alunos.

Bloco 1: Arquitetura da ULA de 32 bits (0:00 – 1:15)

Otimização Estrutural e Complemento de Dois

- **Somador/Subtrator Unificado (Harris & Harris):** Instanciação de apenas 1 somador de 32 bits com inversão condicional:

$$B_{\text{inv}} = B \oplus \text{alu_control}[2], \quad S = A + B_{\text{inv}} + \text{alu_control}[2]$$

- **Flag Zero em Tempo Mínimo:** Implementada via redução unária de baixo atraso lógico:

$$\text{flag_zero} = \sim(|\,\text{alu_result})$$

- **Operação SLT com Sinal Correto:** Utiliza o MSB do resultado somado ao tratamento de overflow.
- **Conjunto Operacional:** `AND`, `OR`, `ADD`, `SUB`, `SLT`, `NOR`.

ROTEIRO DE LOCUÇÃO

Locutor: José Dercy (Líder / DevOps)

"Olá a todos. Sou o José Dercy e apresento a Segunda Entrega do processador MIPS Monociclo da disciplina de ASD da UFAM. Na ULA de 32 bits, adotamos a arquitetura canônica de Harris & Harris: unificamos adição e subtração em uma única cadeia de soma, invertendo B por portas XOR e injetando o carry-in quando o bit 2 de controle está ativo. A flag zero usa redução unária com árvore NOR de baixo atraso e a operação SLT foi desenhada com tratamento formal de números com sinal."

Bloco 1: Verificação da ULA no GTKWave (1:15 – 1:50)

Bateria Formal de Testes Unitários (`tb_alu.v`)

- 11 casos de teste formais com auto-verificação:
  - Operações lógicas bit a bit (`AND`, `OR`, `NOR`);
  - Soma e subtração de positivos e negativos;
  - Detecção imediata de flag zero em resultados nulos;
  - Casos de borda para comparação assinada (`SLT`).
- 100% de aprovação sem advertências de simulação.

AÇÃO DE VÍDEO / GRAVAÇÃO DE TELA

Compartilhar Tela: Terminal & GTKWave

1. No terminal, digitar: `make waves-alu`;
2. O GTKWave abrirá exibindo o sinal `alu.vcd`;
3. Apontar as transições de `alu_control[2:0]`;
4. **Foco na Narração:** Mostrar o momento exato em que a subtração $A - B$ resulta em zero e o sinal `flag_zero` vai para nível alto (1) no mesmo instante.

ROTEIRO DE LOCUÇÃO

"Como vemos no GTKWave, a ULA responde prontamente a cada transição de controle, calculando os resultados e atualizando a flag zero instantaneamente."

Bloco 1: Síntese no Intel Quartus Prime e RTL Viewer (1:50 – 2:30)

Configuração da Síntese (Rubrica 1)

- **Dispositivo FPGA:** Intel MAX 10 (`10M50DAF484C7G`);
- **Placa Alvo:** Terasic DE10-Lite;
- **Arquivos de Projeto:** `quartus/mips_monociclo.qpf` e `.qsf`;
- **Comandos Make:**
  - `make quartus`: Análise e Elaboração rápida;
  - `make quartus-build`: Síntese completa + Fit + Bitstream `.sof`.

AÇÃO DE VÍDEO / GRAVAÇÃO DE TELA

Compartilhar Tela: Intel Quartus Prime 19.1

1. Mostrar o projeto aberto no Quartus Prime;
2. Acessar menu: **Tools** → **Netlist Viewers** → **RTL Viewer**;
3. Navegar até a entidade `alu`;
4. **Foco na Narração:** Apontar o somador de 32 bits sintetizado e os multiplexadores de seleção da operação.

ROTEIRO DE LOCUÇÃO

"Aqui no RTL Viewer do Quartus, observamos a correta inferência do somador unificado e a árvore de multiplexação. Passo agora a palavra ao Vinicius."

Bloco 2: Banco de Registradores de 32 bits (2:30 – 3:45)

Arquitetura do Register File (`reg_file.v`)

- **Dimensões:** 32 registradores de 32 bits ($0 a $31).
- **Topologia de Portas:**
  - 2 portas de leitura combinacional/assíncrona (`rdata1`, `rdata2`);
  - 1 porta de escrita síncrona (`wdata`, `reg_write`).
- **Temporização em Monociclo:** A leitura combinacional permite fornecer operandos para a ULA no mesmo ciclo; a escrita ocorre na borda de subida (`posedge clk`).
- **Garantia Estrutural do Registrador Zero ($0):**

  $$\text{rdata} = (\text{raddr} == 5'b0) \ ? \ 32'b0 : \text{registers}[\text{raddr}]$$

  Imune a qualquer sobrescrita indevida por software.

ROTEIRO DE LOCUÇÃO

Locutor: Vinicius Menezes

"Obrigado, Dercy. Sou o Vinicius e trabalhei no Banco de Registradores e na Unidade de Controle. O Banco de Registradores conta com 32 posições de 32 bits. Para operar em monociclo, a leitura é assíncrona, viabilizando o fluxo contínuo de dados até a ULA em um único ciclo, enquanto a escrita é síncrona na subida do clock. O registrador $0 é travado em zero diretamente por hardware, garantindo total conformidade com o padrão MIPS."

Bloco 2: Unidade de Controle e Decodificação de Saltos (3:45 – 5:00)

Decodificação Hierárquica e Suporte aos Jumps

- **Controle Principal:** Decodifica `Opcode[5:0]` gerando os sinais de controle do datapath (`RegDst`, `ALUSrc`, `MemWrite`, `Branch`, etc.).
- **Tratamento Rigoroso das Instruções de Desvio:**
  - `beq` (Tipo-I): Ativa `Branch`; decisão tomada por `Branch & Zero`;
  - `j` (Tipo-J): Ativa `Jump`; salto para endereço pseudodireto de 26 bits;
  - `jal` (Tipo-J): Ativa `Jump` e sinal `Jal`, gravando $PC + 4$ no registrador \$31 (\$ra);
  - `jr` (Tipo-R): Decodifica `Funct = 0x08`, ativando `JR_Control` para rotear o registrador lido (\$rs) ao próximo PC.

ROTEIRO DE LOCUÇÃO

Locutor: Vinicius Menezes

"A Unidade de Controle foi dividida entre o decodificador principal e o controle da ULA. O ponto central desta Segunda Entrega foi o suporte aos desvios: O desvio condicional `beq` combina o sinal `Branch` com a flag zero da ULA. O salto `j` realiza o desvio incondicional. O `jal` faz a chamada de função salvando $PC + 4$ em \$31. E a instrução `jr` fecha o ciclo de retorno de sub-rotina ao direcionar o valor de \$ra de volta para o Program Counter. Passo a palavra para a Isabela."

Bloco 3: Subsistema de Memória ROM e RAM (5:00 – 6:15)

Arquitetura Harvard no Nível de Módulos

- **Memória de Instruções (`instr_mem.v` – ROM):**
  - 256 palavras de 32 bits (1 KB);
  - Leitura assíncrona alinhada por palavra (`pc[31:2]`);
  - Carga do programa via diretiva `$readmemh`.
- **Memória de Dados (`data_mem.v` – RAM):**
  - 256 palavras de 32 bits para dados voláteis;
  - Leitura combinacional rápida para a instrução `lw`;
  - Escrita síncrona habilitada pelo sinal `mem_write` (`sw`).
- **Eliminação de Conflitos Estruturais:** A separação de barramentos viabiliza busca de instrução e acesso a dados no mesmo ciclo de clock.

ROTEIRO DE LOCUÇÃO

Locutor: Isabela Furtado

"Olá, pessoal, sou a Isabela. Fiquei com o subsistema de memórias e a integração do Top-Level. Adotamos a arquitetura Harvard clássica com memórias separadas de instrução e de dados para eliminar qualquer conflito estrutural no ciclo único. A memória de instruções é uma ROM assíncrona alinhada por palavras, inicializada com o programa de teste. A memória de dados é uma RAM com leitura rápida para a instrução `lw` e escrita síncrona para `sw`."

Bloco 3: Fiação Estrutural do Top-Level e Próximo PC (6:15 – 6:40)

Cascata Ordenada de Multiplexadores para o Próximo PC

A determinação do próximo endereço do PC obedece à hierarquia estrita:

1. **Multiplexador de Branch:**

$$PC_{\text{seq}} = (\text{branch} \wedge \text{zero})\ ?\ PC_{\text{branch}} : PC + 4$$

2. **Multiplexador de Jump (`j` / `jal`):**

$$PC_{\text{j_sel}} = \text{jump}\ ?\ \{PC{+}4[31:28], \text{instr}[25:0], 2'b00\} : PC_{\text{seq}}$$

3. **Multiplexador de JR (`jr`):**

$$PC_{\text{next}} = \text{jr_ctrl}\ ?\ \text{rdata1} : PC_{\text{j_sel}}$$

ROTEIRO DE LOCUÇÃO

Locutor: Isabela Furtado

"Na integração do caminho de dados e top-level, o ponto crítico foi a cascata de seleção do próximo PC. Primeiro avaliamos se há desvio condicional (`beq`). Em seguida, multiplexamos para saltos incondicionais diretos (`j` ou `jal`). E, por fim, o multiplexador prioritário de `jr` comuta para o valor lido do registrador (`$rs`), garantindo o correto retorno de funções."

Bloco 3: Demonstração do Top-Level no GTKWave (6:40 – 7:30)

Cenário de Teste Executado no Top-Level

- 1. Carga de imediatos nos registradores com `addi`;
- 2. Operações lógicas e aritméticas (`add`, `sub`);
- 3. Desvio condicional `beq` tomado e não-tomado;
- 4. Chamada de procedimento com `jal`, salvando $PC + 4$ em \$31;
- 5. Retorno de sub-rotina bem-sucedido via `jr $ra`.

AÇÃO DE VÍDEO / GRAVAÇÃO DE TELA

Compartilhar Tela: GTKWave Top-Level

1. No terminal, executar: `make waves-top`;
2. Apontar no GTKWave os sinais: `pc[31:0]`, `instr[31:0]`, `jump`, `jal`, `jr_ctrl`;
3. **Foco na Narração:** Mostrar o momento exato em que a instrução `jal` é executada, gravando o endereço no registrador \$31, e como a instrução posterior `jr $ra` restabelece o fluxo sem falhas de temporização.

Síntese Física no FPGA Intel MAX 10 e Métricas de Projeto (7:30 – 7:50)

Resultados de Síntese no Intel Quartus Prime 19.1

- **FPGA Alvo:** Intel MAX 10 (`10M50DAF484C7G`) – DE10-Lite;
- **Utilização de Recursos:** 0 erros de compilação (`make quartus-build`);
- **Caminho Crítico da Instrução mais Longa (`lw`):**

$$T_{clk} \geq T_{ROM} + T_{RegFile} + T_{ALU} + T_{RAM} + T_{Mux} + T_{Setup}$$

- **Código 100% Parametrizado:** Declaração rigorosa de parâmetros em todos os módulos HDL.

Governança e DevOps

- **Reprodutibilidade Total:** Makefile com 14 alvos operacionais;
- **Bateria de Testbenches:** Verificação formal cobrindo todos os módulos unitários;
- **Documentação Oficial:** Relatório IEEE compilado via Tectonic.

ROTEIRO DE LOCUÇÃO

Locutor: José Dercy

"A síntese física no Quartus Prime comprovou a viabilidade física do processador na DE10-Lite, com restrições temporais atendidas e governança em DevOps."

Conclusão e Próximos Passos: Rumo ao Pipeline (7:50 – 8:00)

Atendimento Integral das 5 Rubricas

- ✓ **Rubrica 1 – Ambiente:** Fluxo Quartus + Icarus + GTKWave funcional;
- ✓ **Rubrica 2 – Datapath com Jumps:** Saltos `j`, `jal`, `jr`, `beq` operando;
- ✓ **Rubrica 3 – Testbenches:** Bateria completa com 100% de aprovação;
- ✓ **Rubrica 4 – Parametrização:** Módulos Verilog parametrizados;
- ✓ **Rubrica 5 – Vídeo e Artigo:** Roteiro executado e artigo IEEE compilado.

Transição para a Terceira Entrega

Base estrutural pronta para segmentação em 5 estágios com Hazards e Forwarding.

ROTEIRO DE LOCUÇÃO

Locutores: Dercy, Vinicius e Isabela

"Concluímos assim a Segunda Entrega do processador MIPS Monociclo. Obrigado pela atenção do professor e dos colegas! O código e o relatório técnico completo estão disponíveis no nosso repositório no GitHub. Muito obrigado!"

AÇÃO DE VÍDEO / GRAVAÇÃO DE TELA

Câmera Final: Os três alunos em chamada aberta agradecendo e despedindo-se.